Where does OpenSSL fit in?

OpenSSL is a commercial grade open source toolkit, under an Apache-style licence, which according to Wikipedia is implicitly used by two-thirds of all Web users, as of 2014! It is used in everything from quick personal scripts to some of the largest commercial email and Web services.

It provides implementation for Secure Sockets Layer (SSL) v2 and v3, and Transport Layer Security (TLS) v1 protocols, as well as a default software implementation for general-purpose cryptographic algorithms.

OpenSSL is now widely accepted and is applied in various ways in the real world. It is used as a command line utility and as a library that is linked to userland applications. It is used in Perl scripts, and there are open source projects that develop wrappers around the OpenSSL library (like pyOpenSSL, which provides a Python wrapper around the OpenSSL library). But, more importantly, it is used in commercial servers handling millions of SSL/TLS sessions.

Customising OpenSSL

For TLS communication, OpenSSL first establishes a TCP socket communication with the other end over the specified IP address and port number. Then it exchanges handshake messages over this connection to establish a TLS session. It then uses this secure connection to communicate data, as TLS records, with the other end.

OpenSSL provides a library called Libcrypto, which is the default software implementation. This is used for cryptographic operations for TLS and is also exposed in the interface as utility crypto APIs.

The software implementation of Libcrypto would ideally be good enough for many general purpose SSL/TLS clients/servers like browsers, etc, and all general-purpose Libcrypto utility users. However, this may not be enough for many real world applications processing large amounts of traffic. OpenSSL developers understand this real world need and have found a way forward through customisation.

Dedicated hardware helps

Cryptographic operations are typically iterative and involve intense processing. Asymmetric key algorithms like RSA, especially, are used during the initial handshake while establishing a SSL/TLS connection. When processing a large number of TLS connections, for example, on a HTTPS server handling millions of connections, the encryption/decryption being performed in the software on general-purpose processors can be very heavy. This slows down processing on the server, even affecting its core functionality. Dedicated hardware accelerators called Hardware Security Modules (HSMs) are commercially available in various forms, like PCI cards that can be hooked on to be used for faster encryption/decryption. Some modules help in accelerating only certain asymmetric key algorithms like RSA, thus speeding up the initial handshake. Some can help in speeding symmetric key block cipher algorithms like AES, thus speeding up encrypted data throughput. Some can help in both. Based on the requirement, implementers can pick and choose the right hardware acceleration.

Figure 1: SSL traffic growth

Processing speed apart, hardware cryptographic modules are more secure than software implementations. Some standards like FIPS-140 and FIPS 140-2, defined by a US government body called the National Institute of Standards and Technology (*http://nist.gov*), define certain security levels to be maintained by a crypto module. Higher levels of security, like resistance to key disclosure, can be provided only by a hardware module. Based on these standards, certain businesses like banks mandate a higher-level security requirement. This would require external hardware for cryptographic operations.

Using HSMs with OpenSSL

As discussed earlier, OpenSSL is a commercial grade toolkit. It has a very robust, well-written TLS protocol